SCRUM DIGITAL
Agile Solutions For Dynamic Business Requirements

Day 11: Building a Secure Login System

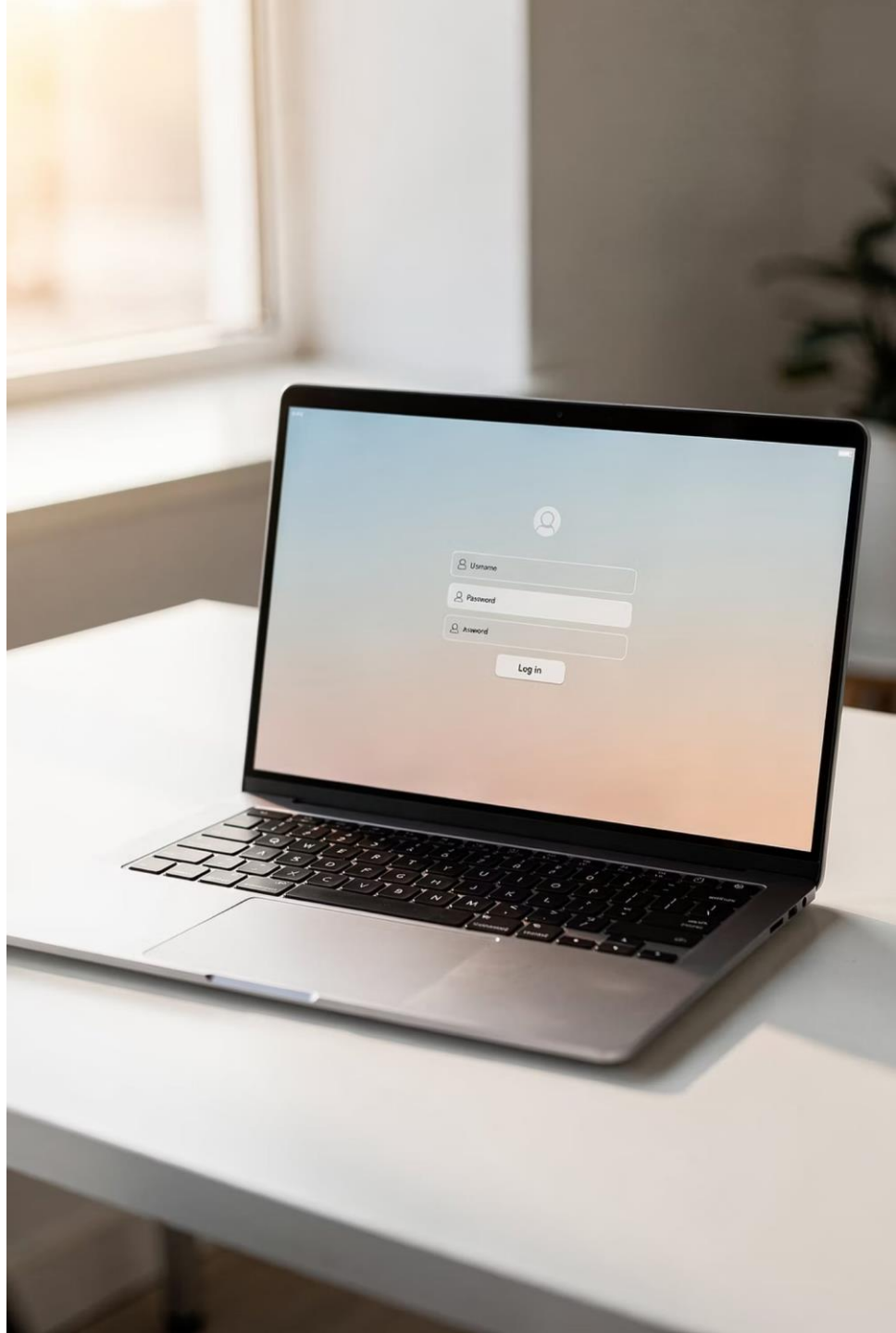
Protecting Your Application — Full Stack Bootcamp

🛠️ Format

Hands-On Workshop

🎯 Goal

Secure Your App



OPENING HOOK

Can Anyone Access Your Student Management System?

🥷 Person A

Opens browser → types your URL → lands directly on the Dashboard.

🥷 Person B

No account. No password. Same result — full access to all student data.

⚠ Should every person on the internet have access to your Student Management System? Right now... they probably do. Let's fix that today.

Your app works — but it's wide open. Any anonymous user can view, edit, or delete student records. Today we close that door, build a login system, and protect every page behind a session.

TODAY'S MISSION

What We're Building Today

By the end of this session, your Student Management System will be fully protected by a real login system. Here's the complete feature set we'll ship together.

🔐 Login Page

A polished Bootstrap login form with email and password fields.

🏠 Protected Dashboard

The dashboard only loads if the user is authenticated via session.

👋 Welcome User

Display the logged-in user's name on the dashboard after authentication.

🚪 Logout

Safely destroy the session and redirect back to the login page.

⚡ **RAPID FIRE RECAP — Pairs Activity (5 min)**

Before We Code — Let's Warm Up

Turn to your partner. Discuss these five questions. You have 60 seconds each. Be ready to share your answers with the class.

1. INSERT vs. UPDATE

What's the difference? When do you use each one?

2. Why Primary Key?

What happens to your table if there's no Primary Key?

3. WHERE Clause

What's the purpose of WHERE? What happens if you forget it in a DELETE?

4. Why Validate?

What could go wrong if you skip form validation entirely?

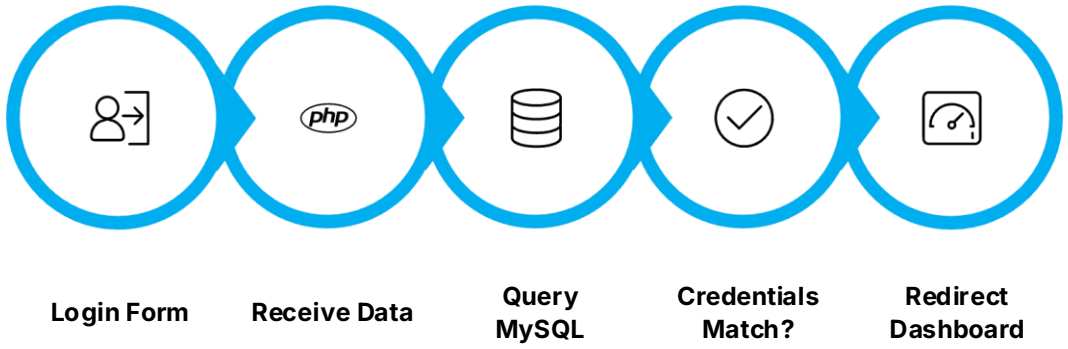
5. Purpose of a Database

Why not just store data in a PHP array instead of MySQL?

MODULE 1

Login System — Concept

A login system has one job: **verify who you are before letting you in**. The flow is straightforward — the user submits a form, PHP checks the credentials against MySQL, and if they match, the user is granted access. No match? Show an error.



Step 1

User fills login form with email + password

Step 2

PHP receives `$_POST` data

Step 3

Run SQL: `SELECT * FROM users WHERE email = ?`

Step 4

Row found + password matches → access granted

Keep it simple. Username + password. One SQL query. One redirect. That's the entire concept.

MODULE 1 — LIVE CODING

Follow Along in VS Code

Building the Login Form

Open VS Code. Create `login.php`. We'll use Bootstrap 5 to build a clean, centered login card. Here's the structure we're coding right now:

```
<!-- login.php -->
<form method="POST" action="login.php">
  <div class="mb-3">
    <label>Email Address</label>
    <input type="email" name="email"
           class="form-control"
           placeholder="you@example.com">
  </div>
  <div class="mb-3">
    <label>Password</label>
    <input type="password" name="password"
           class="form-control">
  </div>
  <button type="submit"
          class="btn btn-primary w-100">
    Login
  </button>
</form>
```

🎯 MISSION 1

Create a Bootstrap Login Page with the following fields:

- Email Address input
- Password input
- Login button (full width)
- Card centered on the page

🏆 CHALLENGE

Display an "Invalid Credentials" Bootstrap Alert in red when the form is submitted empty.

⭐ BONUS

Add a **Show Password** toggle button using a small JavaScript snippet.

MODULE 2

Authentication — Concept

Authentication means **confirming the user is who they claim to be**. We do this by looking up their email in the database, then checking if the stored password matches what they typed. If both match — they're in.

1

Valid Login

Email ✓ + Password ✓ → Redirect to Dashboard

2

Invalid Login

No match found → Show error message, stay on login page

The SQL query we use:

```
<?php
$email = $_POST['email'];
$sql = "SELECT * FROM users
        WHERE email = '$email'
        LIMIT 1";
$result = mysqli_query($conn, $sql);
$user = mysqli_fetch_assoc($result);

if ($user &&
    $user['password'] == $_POST['password']) {
  // Success!
  header("Location: dashboard.php");
} else {
  $error = "Invalid credentials.";
}
?>
```

ⓘ We'll improve password security with hashing in a future session. For now, keep it simple and focus on the flow.

MODULE 2 — MISSION + CHALLENGE

Authenticate Your Users

Create the `users` table in MySQL

Fields: `id`, `name`, `email`, `password`

Write the PHP authentication logic

Query the database with the submitted email. Check password. Redirect on success.

Test both paths

Valid credentials → Dashboard. Invalid → Error message stays visible.

🎯 MISSION 2

Authenticate users against your MySQL `users` table. On success, redirect to `dashboard.php`. On failure, show a red Bootstrap Alert.

🏆 CHALLENGE

After a successful login, display: **"Welcome, [Student Name]!"** on the dashboard using the name from the database.

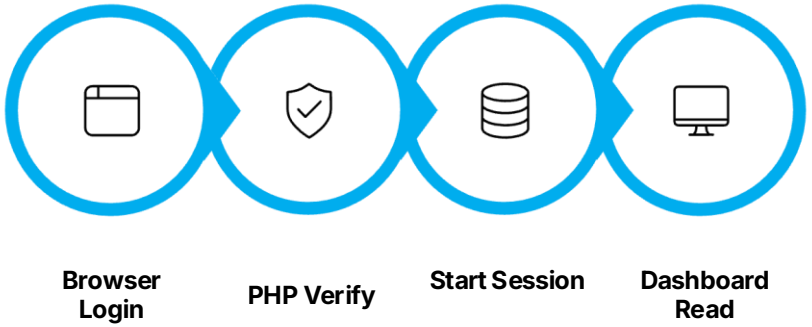
⭐ BONUS

Store the last login timestamp in the database. Display it on the dashboard: *"Last login: Dec 10, 2024 at 2:34 PM"*

MODULE 3

Sessions — Concept

A **session** is PHP's way of remembering a logged-in user across multiple pages. Without sessions, the user would have to log in on every single page. With sessions, PHP creates a unique ID stored in the browser, and ties it to data stored on the server.



Two key tools: `session_start()` must be called at the top of **every PHP page** that uses sessions. `$_SESSION` is the superglobal array where you store and read session data.

```php
<?php
// After successful login:
session_start();

$_SESSION['user_id']   = $user['id'];
$_SESSION['user_name'] = $user['name'];
$_SESSION['user_email']= $user['email'];

header("Location: dashboard.php");
exit();
?>
```

```php
<?php
// On dashboard.php:
session_start();

echo "Welcome, "
   . $_SESSION['user_name']
   . "!";
?>
```

Always call `session_start()` before any HTML output — even a single blank line before `<?php` will break it.

MODULE 3 — MISSION + CHALLENGE

Create a Session After Login

Now it's your turn. After a successful login, start a session and store the user's data. Then read that data on the dashboard page to confirm the session is working correctly.

1

Call `session_start()`

Add it to the very top of both `login.php` and `dashboard.php`

2

Store session data

Save `user_id`, `user_name`, and `email` into `$_SESSION`

3

Read on dashboard

Echo the user's name from `$_SESSION['user_name']`

🎯 MISSION 3

After a successful login, start a session and store the user's name, ID, and email. Navigate to the dashboard — confirm you can display the stored session data.

🏆 CHALLENGE

Display a personalized greeting on the dashboard: **"Welcome back, Mohit! 👏"**

⭐ BONUS

Store and display the login timestamp: **"Session started at 2:45 PM"**

MODULE 4

Protected Pages — Concept

Protecting a page means **checking for a valid session before showing any content**. If the session exists, load the page. If not, immediately redirect the user back to the login page. This check must happen at the very top of every protected page — before any HTML is rendered.

```php
<?php
session_start();

// Guard clause – protect this page
if (!isset($_SESSION['user_id'])) {
  header("Location: login.php");
  exit(); // Always exit after
redirect!
}
// Safe to render the page now
?>
<!DOCTYPE html>
<html>
  ...dashboard content here...
</html>
```

Always call `exit()` immediately after `header("Location: ...")`. Without it, PHP continues executing the rest of the page even though the browser redirects.

MODULE 4 — MISSION + CHALLENGE

Protect Your Dashboard

This pattern — **check session → redirect or proceed** — is the core of every protected web application. You'll use it on every single page of your Student Management System.

🎯 MISSION 4

Add a session guard to `dashboard.php`. Open the URL directly without logging in — confirm you get redirected to `login.php` automatically.

🏆 CHALLENGE

Test all protected pages: try accessing `students.php`, `add_student.php`, and `edit_student.php` directly. Every one should redirect unauthorized users.

⭐ BONUS

Build a friendly **"Access Denied"** page (`403.php`) with a Bootstrap card, a lock icon, and a "Back to Login" button.

MODULE 4 — LOGOUT

Logging Out — Clean Session Destroy

Logout means **destroying the session completely** so no trace of the user's data remains on the server. A proper logout does three things: unsets all session variables, destroys the session, and redirects to login.

```php
<?php
// logout.php
session_start();

// Step 1: Clear all session data
$_SESSION = array();

// Step 2: Destroy the session
session_destroy();

// Step 3: Send user back to login
header("Location: login.php");
exit();
?>
```

Add a logout button to your navigation bar. Link it to `logout.php`:

```html
<!-- navbar logout button -->
<a href="logout.php"
 class="btn btn-outline-danger btn-sm">
```

🐛 BUG HUNT

Spot the Bug — Broken Login System

The code below is a login system with **5 bugs** hidden in it. Work in pairs. Find them all before the reveal. Go!

```php
<?php
// login.php — Find the bugs!

$email    = $_POST['email'];
$password = $_POST['password'];

$sql = "SELECT * FROM users
        WHERE email = '$email'";
$result = mysqli_query($conn, $sql);
$user   = mysqli_fetch_assoc($result);

if ($user['password'] == $password) {
  $_SESSION['user_id'] = $user['id'];
  header("Location: dashboard.php");
}

echo "Invalid credentials.";
?>
```

🔍 Bugs to Find:

Bug 1

Missing `session_start()` at the top of the file

Bug 2

No `exit()` after the redirect — page keeps executing

Bug 3

SQL returns multiple rows — no `LIMIT 1`

Bug 4

Error message always shows — even on a successful login

Bug 5

No database connection (`$conn`) included or defined

🚀 BUILD SPRINT — 30 MINUTES

Secure Your Student Management System

Requirements Checklist

🔐 Login Page

Bootstrap form with email + password

🔑 Authentication

Verify credentials against MySQL

🧠 Sessions

Start session, store user data after login

🛡️ Protected Pages

Guard every CRUD page with session check

🚪 Logout

Destroy session and redirect to login

⏱️ Timer: 30 Minutes — Build as much as you can. Bonus points for a polished UI and showing the logged-in user's name in the navbar.

⭐ Bonus Goals

- Show logged-in user's name in the top navbar
- Professional Bootstrap UI with sidebar navigation
- Friendly "Access Denied" page for unauthorized access
- Display last login time on the dashboard card

Files You'll Need

- `login.php` — form + authentication
- `dashboard.php` — protected home page
- `logout.php` — session destroy
- `db.php` — database connection (include on all pages)

🎤 DEMO DAY

Student Showcase — Present Your Work

Each team gets **2 minutes** on stage. Walk us through your working login system. Be proud — you built this from scratch today.

Present These 4 Things:

Login

Show a successful login with real credentials from your database

Dashboard

Show the protected dashboard with the user's name displayed

Logout

Click logout — confirm you're redirected and session is gone

Biggest Challenge

Share one bug you hit and how you fixed it

Trainer will ask each team one authentication question. Be ready!

🧠 INTERVIEW QUIZ

5 Questions — Answer Without Notes

These are real interview questions. Answer them out loud with your partner first, then we'll discuss as a class.

1

What is Authentication?

Define it in one sentence. What exactly does PHP verify during login?

2

Why Use Sessions?

What problem do sessions solve? What would happen without them?

3

Login vs. Session

What's the difference between the login process and a session? Are they the same thing?

4

What Happens After Logout?

Walk through exactly what `logout.php` does, step by step.

5

Predict the Output

If `session_start()` is missing from `dashboard.php`, what happens when a logged-in user visits it?

📋 ASSIGNMENT

Take It Further — Improve Your Project

You have a working secure login system. Now push it further. These improvements will make your project look **production-ready** — the kind of portfolio piece that impresses in interviews.

Forgot Password UI

Build the UI for a forgot password flow — form with email input and a confirmation message. (No email sending required — just the interface.)

Profile Picture

Add a profile picture column to the `users` table. Display the avatar in the navbar and on a profile page.

Change Password UI

Build a "Change Password" form that accepts current password, new password, and confirm password fields.

Push to GitHub

Commit all changes with a meaningful message and push to your GitHub repo. Share the link with your trainer.

🏁 DAY RECAP

Today's Achievement — You Built This

✅ Login Page

Bootstrap form, POST submission, clean UI

✅ Authentication

SQL query against MySQL users table

✅ Sessions

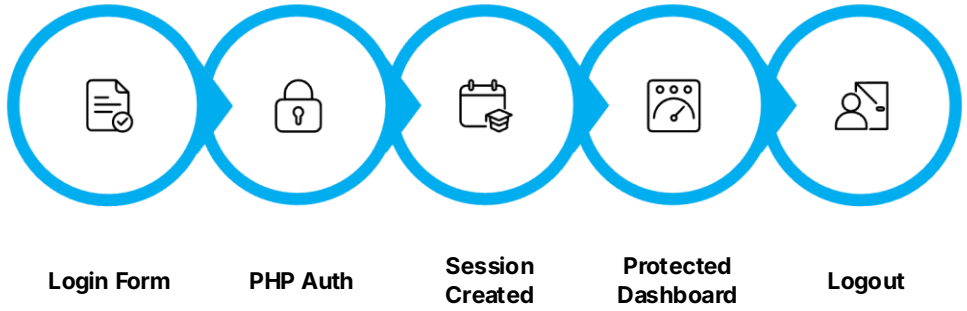
`session_start()`, `$_SESSION`, user data stored

✅ Protected Pages

Session guard on every CRUD page

✅ Logout

Session destroyed, redirect to login



> "Before today, your Student Management System was open to the world. Right now, it's protected. You built that from scratch in under 3 hours. That's real engineering."

COMING TOMORROW — DAY 12 OF 13

Making Your Project Production Ready

🔍 Search

Add live search to your student list — filter records in real time

📄 Pagination

Handle large datasets gracefully with MySQL `LIMIT` and `OFFSET`

📊 Dashboard

Add stats cards — total students, recent additions, summary metrics

✨ Final Polish

Professional navigation, responsive layout, portfolio-ready finish

Today you secured your application. Tomorrow you'll transform it into a professional project you can proudly show to any employer. 🚀